

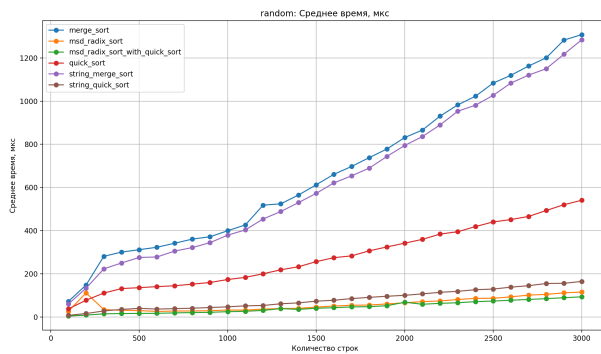
Отчет

Ссылки и посылки

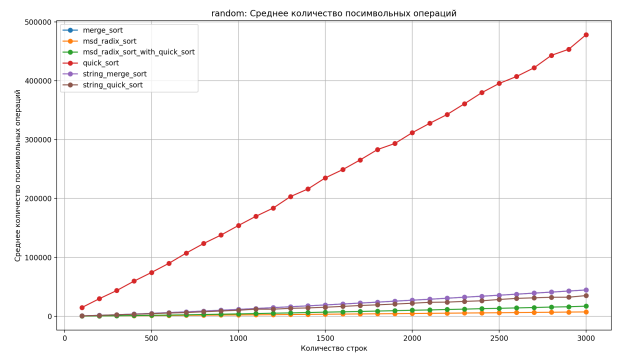
GitHub: <https://github.com/karnaukhovGA69/SET9-69>

Задача	Посылка
A1m	375980010
A1q	375980021
A1r	375980403
A1rq	375980411

Графики

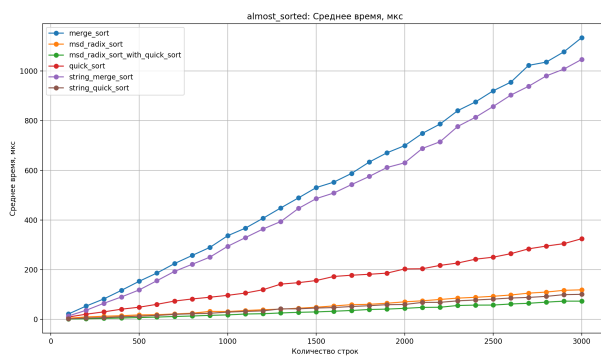


(a) Среднее время работы

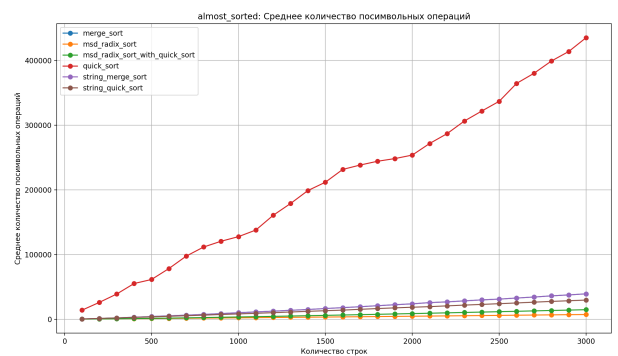


(b) Среднее число посимвольных операций

Рис. 1: Случайный набор строк

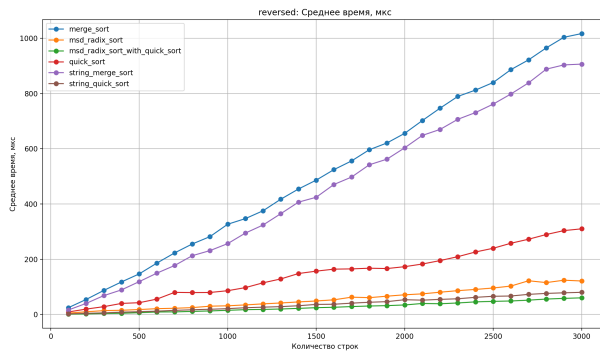


(a) Среднее время работы

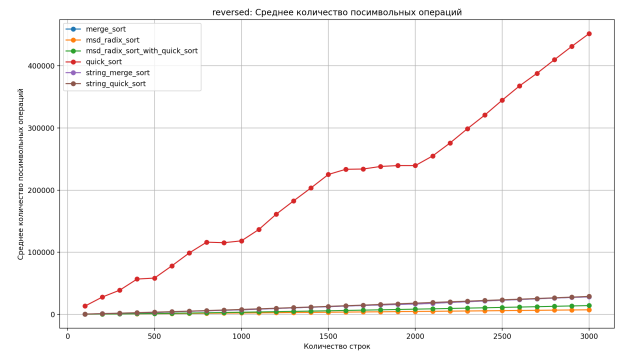


(b) Среднее число посимвольных операций

Рис. 2: Почти отсортированный набор строк



(а) Среднее время работы



(б) Среднее число посимвольных операций

Рис. 3: Обратно отсортированный набор строк

Анализ результатов

1. Время работы.

- На наборах `random`, `almost_sorted` и `reversed` самым быстрым оказался гибридный `MSD Radix Sort` с переходом на `QuickSort` для малых подмассивов. Он стабильно выигрывает по времени и заметно опережает остальные алгоритмы на больших входных данных.
- На тех же наборах обычный `MSD Radix Sort` чаще всего занимает второе или третье место. По времени он уступает гибридной версии, но все равно выглядит быстрее стандартных сравнительных сортировок.
- На строках с длинными общими префиксами лучший средний результат по времени показывает алгоритм *Ternary String QuickSort*.
- На больших входных данных стандартные `QuickSort` и `MergeSort` заметно уступают модифицированным строковым алгоритмам. Текущая реализация *String MergeSort* по времени тоже не дает преимущества.

2. Количество посимвольных операций.

- На всех рассмотренных наборах минимальное число посимвольных проверок показывает обычный `MSD Radix Sort`. По данным эксперимента он остается лидером и на случайных строках, и на почти отсортированных, и на обратно отсортированных массивах.
- На тех же тестах гибридная версия работает быстрее, но из-за этого страдает число сравнений. При переключении на `QuickSort` на маленьких подмассивах она по операциям немного проигрывает чистому `MSD`.
- На всех наборах стандартный `QuickSort` дает самый высокий показатель посимвольных сравнений с большим отрывом. При полном сравнении строк его стоимость растет намного быстрее остальных алгоритмов.
- В текущих результатах *String MergeSort* не показывает преимущества относительно обычного `MergeSort`. Их кривые по числу операций почти совпадают, поэтому эффекта от оптимизации здесь не видно.